ABSTRACT

The Student Attendance Management System is a simple web-based application designed to maintain student attendance records digitally. In a traditional classroom, attendance is often recorded manually, which can consume time and make record maintenance difficult. The proposed system provides a computerized solution for storing student information, marking daily attendance and viewing attendance records.

The application is developed using Python with the Flask framework for the backend, HTML and CSS for the user interface, and SQLite for database storage. The system allows a user to add student details, mark students as Present or Absent for a selected date, view attendance records and calculate attendance percentage.

The project demonstrates the basic concepts of web application development, database management, CRUD operations, routing, form handling and data processing. It is intentionally designed as a beginner-friendly B.Tech CSE mini project so that each module can be understood and explained easily.

Keywords: Attendance Management, Python, Flask, SQLite, HTML, CSS, Web Application, Database.

TABLE OF CONTENTS

Chapter	Title
1	Introduction
2	Problem Statement and Objectives
3	Existing and Proposed System
4	Feasibility Study
5	System Requirements
6	System Design
7	Database Design
8	Implementation
9	Testing
10	Results and Discussion
11	Advantages, Limitations and Future Scope
12	Conclusion
13	References
Appendix A	Viva Questions and Answers

1. INTRODUCTION

Attendance is an important part of academic administration. Colleges and universities need accurate records to monitor student participation and calculate attendance percentages. When attendance is maintained using paper registers or scattered spreadsheets, it can require significant manual effort.

The Student Attendance Management System solves this basic problem by providing a small web application in which student records and attendance information are stored in a database. The application provides a dashboard, student registration, attendance marking and attendance reporting.

1.1 Background

Digital record management is useful because information can be stored in a structured form and retrieved quickly. A database also makes it easier to calculate statistics such as the number of present days and attendance percentage.

1.2 Purpose

The purpose of this project is to develop a simple, low-cost attendance system that demonstrates fundamental programming, web development and database concepts.

1.3 Scope

- Add and store student details.
- Mark daily attendance for registered students.
- View attendance records.
- Calculate attendance percentage.
- Delete student records when required.
- Provide a simple dashboard for quick information.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Manual attendance systems require teachers to maintain registers and calculate attendance percentages manually. Records may become difficult to search, update or summarize. Therefore, a simple computerized system is required to store student details and attendance records in an organized database.

2.2 Objectives

- To create a simple web-based attendance application.
- To store student details in a database.
- To mark daily attendance as Present or Absent.
- To prevent duplicate attendance for the same student and date.
- To display attendance history.
- To calculate attendance percentage automatically.
- To demonstrate Python, Flask, HTML, CSS and SQLite integration.

2.3 Project Outcomes

After implementation, the user can manage student records and attendance from a browser. The project also provides practical understanding of client-server interaction, database tables, SQL queries and basic application routing.

3. EXISTING AND PROPOSED SYSTEM

3.1 Existing System

In the traditional method, attendance is recorded in a paper register or a basic spreadsheet. The teacher manually searches student names, marks attendance and later calculates percentages. This approach is simple but can be time-consuming.

3.2 Drawbacks of Existing System

- Manual calculation takes time.
- Searching old attendance records can be difficult.
- Paper records may be lost or damaged.
- Duplicate or inconsistent entries can occur.
- Generating attendance statistics requires extra work.

3.3 Proposed System

The proposed system stores student and attendance data in SQLite. A web interface allows the user to add students and mark attendance. The application automatically retrieves records and calculates attendance percentages.

3.4 Comparison

Feature	**Manual System**	**Proposed System**
Student records	Register	Database
Attendance marking	Manual	Web form
Calculation	Manual	Automatic
Search	Difficult	Quick
Record storage	Paper/Spreadsheet	SQLite
Duplicate control	Manual	Database constraint

4. FEASIBILITY STUDY

4.1 Technical Feasibility

The project is technically feasible because it uses freely available technologies such as Python, Flask, HTML, CSS and SQLite. The application can run on an ordinary computer without requiring a dedicated server.

4.2 Economic Feasibility

The project has very low development cost because the software technologies are open source and SQLite does not require a separate database server. Existing college computers can be used for development and demonstration.

4.3 Operational Feasibility

The system is easy to operate because it provides simple navigation and forms. A teacher or administrator with basic computer knowledge can use the main features without extensive training.

4.4 Schedule Feasibility

A basic version can be developed in stages: requirement analysis, database design, interface development, backend integration, testing and documentation.

5. SYSTEM REQUIREMENTS

5.1 Hardware Requirements

Component	Minimum Requirement
Processor	Dual-core processor or better
RAM	4 GB or more
Storage	At least 500 MB free space
Display	Standard monitor
Input	Keyboard and mouse

5.2 Software Requirements

Software	Purpose
Python 3.x	Programming language
Flask	Web framework
SQLite	Database
HTML5	Page structure
CSS3	Page styling
Web Browser	Run the application
VS Code / any IDE	Development

5.3 Functional Requirements

- The system shall add a new student.
- The system shall store student information.
- The system shall mark attendance for a selected date.
- The system shall update attendance if the same date is submitted again.
- The system shall show attendance records.
- The system shall calculate attendance percentage.
- The system shall delete a student and related attendance records.

5.4 Non-Functional Requirements

- Usability: simple navigation and forms.
- Reliability: database constraints reduce duplicate entries.
- Maintainability: modules are separated into templates and backend routes.
- Performance: suitable for a small classroom dataset.

6. SYSTEM DESIGN

6.1 System Architecture

The application follows a simple three-part structure: the browser provides the user interface, Flask processes requests and business logic, and SQLite stores persistent data.

[User / Browser] → [Flask Application] → [SQLite Database]

6.2 Data Flow Diagram – Level 0

[Teacher/Admin] → [Attendance Management System] → [Attendance Database]

The teacher or administrator sends student and attendance information to the system. The system processes the information and stores or retrieves data from the database.

6.3 Data Flow Diagram – Level 1

Student Details → Student Module → Students Table

Attendance Input → Attendance Module → Attendance Table

Database Records → Report Module → Attendance Report

6.4 Use Cases

Actor	Use Case
Teacher/Admin	Add student
Teacher/Admin	View students
Teacher/Admin	Mark attendance
Teacher/Admin	View attendance
Teacher/Admin	View percentage
Teacher/Admin	Delete student

7. DATABASE DESIGN

SQLite is used because it is lightweight and suitable for a small educational application. The database contains two main tables: students and attendance.

7.1 Students Table

Field	Type	Key	Description
id	INTEGER	Primary Key	Unique student record ID
roll_no	TEXT	Unique	Student roll number
name	TEXT	-	Student name
course	TEXT	-	Course name
year	TEXT	-	Academic year

7.2 Attendance Table

Field	Type	Key	Description
id	INTEGER	Primary Key	Unique attendance ID
student_id	INTEGER	Foreign Key	References students.id
attendance_date	TEXT	-	Attendance date
status	TEXT	-	Present or Absent

7.3 Relationship

STUDENTS (1) ——————< (M) ATTENDANCE

One student can have many attendance records. The student_id field connects each attendance record to a student.

7.4 Attendance Formula

Attendance % = (Present Days / Total Attendance Days) × 100

8. IMPLEMENTATION

8.1 Project Structure

File/Folder	Purpose
app.py	Flask application, routes and database operations
templates/	HTML page templates
static/style.css	User interface styling
attendance.db	SQLite database created automatically
requirements.txt	Python dependency list
PROJECT_REPORT.md	Report content
VIVA_QUESTIONS.md	Viva preparation

8.2 Main Modules

A. Dashboard

Displays total registered students, present students for the current date and a list of registered students.

B. Student Registration

Accepts roll number, name, course and academic year. Roll number is unique so duplicate student records are avoided.

C. Attendance Marking

Displays all registered students and provides Present/Absent choices for a selected date. The database uses a unique student-date combination.

D. Attendance Records

Retrieves attendance by joining the students and attendance tables and displays roll number, name, date and status.

E. Attendance Percentage

Counts present and total attendance records for each student and calculates the percentage automatically.

8.3 Basic Algorithm

- Start application.
- Initialize SQLite database and tables if they do not exist.
- Add or retrieve student records.
- For attendance, select a date and choose Present/Absent for each student.
- Save attendance in the database.
- Retrieve records for reporting.
- Calculate percentage for each student.
- Display result.
- Stop.

9. TESTING

Testing was planned to verify that each major function works correctly and handles common user inputs.

Test ID	Test Case	Input	Expected Result	Status
TC01	Open application	Browser URL	Dashboard opens	Pass
TC02	Add student	Valid details	Student added	Pass
TC03	Duplicate roll number	Existing roll no.	Error message	Pass
TC04	Mark attendance	Date + status	Attendance saved	Pass
TC05	Repeat same date	Same student/date	Existing record updated	Pass
TC06	View records	Attendance menu	Records displayed	Pass
TC07	Percentage	Attendance records	Correct percentage shown	Pass
TC08	Delete student	Student ID	Student and attendance removed	Pass
TC09	Empty form	Missing field	Validation/message shown	Pass

9.1 Validation

Required fields are used in the HTML forms. The backend also checks that important fields are present. SQLite uniqueness constraints protect against duplicate roll numbers and duplicate student-date attendance entries.

9.2 Error Handling

The application shows user-friendly flash messages for successful operations and common errors such as duplicate roll numbers.

10. RESULTS AND DISCUSSION

The implemented system provides a working browser-based interface for basic attendance management. The dashboard provides a quick overview, while separate pages handle student registration, attendance marking and reporting.

10.1 Expected Screens

Figure 1 – Dashboard: Insert a screenshot of the home/dashboard page.

Figure 2 – Add Student: Insert a screenshot showing the student registration form.

Figure 3 – Mark Attendance: Insert a screenshot showing Present/Absent options.

Figure 4 – Attendance Records: Insert a screenshot of saved attendance.

Figure 5 – Student Percentage: Insert a screenshot showing calculated attendance percentage.

10.2 Sample Result

Roll No.	Name	Total Days	Present	Attendance
101	Rahul	10	9	90%
102	Aman	10	8	80%
103	Priya	10	10	100%

The sample result demonstrates how attendance percentage can be calculated from stored records.

10.3 Discussion

The system is suitable for a mini-project demonstration and small classroom datasets. It focuses on core concepts rather than advanced enterprise features, making the implementation easy to understand and extend.

11. ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

11.1 Advantages

- Simple and easy to use.
- Reduces manual attendance work.
- Stores records digitally.
- Automatically calculates attendance percentage.
- Uses a lightweight database.
- Easy to extend with additional modules.

11.2 Limitations

- The current version does not use face recognition.
- There is no role-based login system.
- SQLite is intended for small-scale use in this project.
- Reports are displayed in the browser rather than exported automatically.

11.3 Future Scope

- Add teacher/admin login.
- Add face recognition using OpenCV.
- Export attendance to Excel or PDF.
- Add monthly and subject-wise reports.
- Add email/SMS notifications.
- Move the database to MySQL or a cloud database.
- Deploy the system on a college server.

12. CONCLUSION

The Student Attendance Management System successfully demonstrates how a simple web application can be used to digitize classroom attendance. The project combines Python Flask for application logic, HTML and CSS for the interface, and SQLite for data storage.

The system provides essential functions such as student registration, attendance marking, attendance history and percentage calculation. It reduces the effort required for maintaining manual records and provides faster access to information.

As a B.Tech CSE mini project, the system also provides practical experience in web development, database design, SQL operations, form handling, routing and software testing. The project can be extended in future with biometric recognition, authentication, reporting and cloud deployment.

12.1 Learning Outcomes

- Understanding Flask routing and request handling.
- Creating and querying a relational database.
- Connecting a Python application with SQLite.
- Designing simple web pages using HTML and CSS.
- Implementing CRUD-style operations.
- Writing and executing basic software test cases.

13. REFERENCES

- Python Documentation – Python language reference and standard library.
- Flask Documentation – Flask web framework documentation.
- SQLite Documentation – SQLite database documentation.
- MDN Web Docs – HTML and CSS reference.
- General database and web application development textbooks/lecture notes used during project study.

13.1 Software Used

Tool	**Use**
Python	Backend programming
Flask	Web application framework
SQLite	Database management
HTML/CSS	Frontend
Web Browser	Testing and demonstration
Code Editor	Writing and managing source code